

1. **What is an operating system (OS)?**

Software managing hardware resources and providing services to applications.

2. **Why do we need an OS?**

To abstract hardware complexity and provide efficient resource management.

3. **What is process in OS?**

Program in execution with its own address space, PCB, and state.

4. **What is a thread?**

Lightweight process sharing address space but having own execution context.

5. **Difference between process and thread?**

Process = separate memory; Thread = shares process memory.

6. **What is process state?**

Running, Ready, Waiting, New, Terminated stages of a process.

7. **What is PCB?**

Process Control Block stores process info like ID, state, registers.

8. **What is context switching?**

Saving current process state and loading next process state.

9. **Why context switching is needed?**

To enable multitasking among multiple processes.

10. **What is multiprogramming?**

Running multiple programs to maximize CPU utilization.

11. **What is time-sharing?**

CPU allocated to processes in fixed time slices for responsiveness.

12. **What is preemptive scheduling?**

OS can interrupt running process to assign CPU to higher-priority process.

13. **What is non-preemptive scheduling?**

Process runs until completion or blocking; no forced interruption.

14. **Difference between preemptive and non-preemptive scheduling?**

Preemptive = CPU can be taken; Non-preemptive = CPU held until done.

15. **What is FCFS scheduling?**

First-Come-First-Serve executes processes in arrival order.

16. What is SJF scheduling?

Shortest Job First executes process with minimum burst time first.

17. Difference between SJF preemptive and non-preemptive?

Preemptive = can switch if shorter job arrives; Non-preemptive = executes current fully.

18. What is Round Robin scheduling?

Each process gets CPU for fixed time quantum cyclically.

19. How does RR prevent starvation?

Every process gets a fair time slice in circular order.

20. What is priority scheduling?

CPU assigned based on process priority level.

21. What is starvation?

Low-priority process waits indefinitely for CPU.

22. How to avoid starvation?

Use aging to gradually increase priority of waiting processes.

23. What is multilevel queue scheduling?

Processes divided into queues based on priority or type.

24. What is multilevel feedback queue?

Processes can move between queues based on behavior.

25. What is CPU-bound vs I/O-bound process?

CPU-bound = needs more computation; I/O-bound = waits more for I/O.

26. What is process synchronization?

Coordination to ensure correct execution order among processes.

27. What is critical section?

Part of code accessing shared resources needing mutual exclusion.

28. What is mutual exclusion?

Only one process can access critical section at a time.

29. How is mutual exclusion implemented?

Using locks, semaphores, monitors, or disabling interrupts.

30. What is semaphore?

Integer variable used for signaling and process synchronization.

31. Types of semaphore?

Binary semaphore = 0/1; Counting semaphore = integer counter.

32. What is deadlock?

Processes wait indefinitely for resources held by each other.

33. Necessary conditions for deadlock?

Mutual exclusion, hold and wait, no preemption, circular wait.

34. How to prevent deadlock?

Break one condition, e.g., eliminate circular wait using ordering.

35. How to avoid deadlock?

Use Banker's algorithm to ensure safe allocation.

36. How to detect deadlock?

Track resource allocation graph for cycles.

37. What is resource allocation graph?

Diagram showing processes and resource requests/allocations.

38. What is paging?

Memory divided into fixed-size pages for efficient allocation.

39. What is segmentation?

Memory divided based on logical segments (code, data, stack).

40. Difference between paging and segmentation?

Paging = fixed size, no logical view; Segmentation = variable, logical.

41. What is virtual memory?

Allows processes to use more memory than physically available using disk.

42. What is demand paging?

Load pages into memory only when required.

43. What is page fault?

When required page not in memory, triggering page load from disk.

44. What is page replacement?

Removing a page from memory to load a new page.

45. Algorithms for page replacement?

FIFO, LRU, Optimal, Clock.

46. **What is thrashing?**

Excessive page swapping leading to low CPU utilization.

47. **How to prevent thrashing?**

Use working set model to allocate enough frames to processes.

48. **What is working set?**

Set of pages a process actively uses during execution.

49. **What is segmentation fault?**

Process tries to access memory not assigned to it.

50. **What is I/O bound vs CPU bound scheduling impact?**

I/O-bound benefits RR; CPU-bound benefits SJF or priority.

51. **What is aging in scheduling?**

Gradually increasing priority of long-waiting processes.

52. **What is multithreading?**

Multiple threads within a single process executing concurrently.

53. **Benefits of multithreading?**

Resource sharing, faster response, better CPU utilization.

54. **What is user-level thread?**

Managed by user-level library; OS unaware of thread existence.

55. **What is kernel-level thread?**

Managed by OS; can run on multiple processors simultaneously.

56. **Difference between user and kernel thread?**

User = fast, less control; Kernel = slower, fully managed.

57. **What is process scheduling criteria?**

CPU utilization, throughput, turnaround, waiting, response time.

58. **What is turnaround time?**

Total time from submission to completion of process.

59. **What is waiting time?**

Time a process spends in ready queue.

60. **What is response time?**

Time from submission to first execution.

61. **What is throughput?**

Number of processes completed per unit time.

62. **What is CPU utilization?**

Percentage of time CPU is actively processing.

63. **What is cooperative scheduling?**

Processes voluntarily yield CPU after completion.

64. **What is preemptive scheduling advantage?**

Better responsiveness and avoids process monopolization.

65. **What is the difference between cooperative and preemptive?**

Cooperative = voluntary yield; Preemptive = OS can force switch.

66. **What is starvation in priority scheduling?**

Low-priority processes never get CPU.

67. **What is aging technique?**

Gradually increase priority of waiting process to avoid starvation.

68. **What is thread pool?**

Pre-created set of threads reused for executing tasks.

69. **Advantages of thread pool?**

Reduces overhead of thread creation and improves performance.

70. **What is process communication?**

Exchange of data between processes via shared memory or message passing.

71. **What is inter-process communication (IPC)?**

Mechanism for processes to communicate and synchronize.

72. **What is shared memory IPC?**

Processes communicate by reading/writing common memory segment.

73. **What is message passing IPC?**

Processes communicate by sending/receiving messages.

74. **Synchronous vs Asynchronous message passing?**

Synchronous = sender waits; Asynchronous = sender continues immediately.

75. **What is kernel mode vs user mode?**

Kernel = full access to hardware; User = restricted execution.

76. Why separate kernel and user mode?

Protect OS and system resources from user programs.

77. What is system call?

Interface for programs to request OS services.

78. Examples of system calls?

fork(), exec(), read(), write(), wait().

79. What is fork()?

Creates a new process (child) duplicating parent.

80. What is exec()?

Replaces current process memory with new program.

81. Difference between fork and exec?

fork = duplicate; exec = replace process image.

82. What is zombie process?

Child completed, parent didn't read exit status.

83. What is orphan process?

Child process whose parent terminated.

84. How are orphan processes handled?

Adopted by init process in Unix/Linux.

85. What is process synchronization problem?

Correct ordering of access to shared resources.

86. What is producer-consumer problem?

Processes share buffer; producer produces, consumer consumes.

87. Solution for producer-consumer problem?

Use semaphores to manage empty/full buffer states.

88. What is reader-writer problem?

Multiple readers can read simultaneously, writers exclusive.

89. How to solve reader-writer problem?

Semaphores or mutex with priority to readers/writers.

90. What is dining philosophers problem?

Demonstrates deadlock and resource sharing issues.

91. How to solve dining philosophers problem?

Limit concurrent eating or impose resource ordering.

92. What is real-time OS (RTOS)?

OS guaranteeing response within strict timing constraints.

93. Hard vs soft real-time system?

Hard = missing deadline fails; Soft = missing tolerable.

94. What is interrupt?

Signal to CPU to stop current execution for high-priority task.

95. Hardware vs software interrupt?

Hardware = external device; Software = program-generated.

96. What is trap?

Synchronous software interrupt for OS service.

97. What is polling?

CPU repeatedly checks device status instead of being interrupted.

98. Advantages of interrupt over polling?

Efficient CPU utilization, faster response to events.

99. What is multilevel feedback queue scheduling?

Processes move between queues based on execution characteristics.

100. Difference between process and program?

Program = static code; Process = executing instance with state.

101. What is load balancing in OS?

Distributing processes among processors for efficiency.

102. What is symmetric multiprocessing (SMP)?

Multiple CPUs share common memory and OS.

103. What is asymmetric multiprocessing (AMP)?

One master CPU controls OS, others handle tasks.

104. Advantages of SMP over AMP?

Better fault tolerance, load distribution, and scalability.

105. What is race condition?

Incorrect behavior due to concurrent process timing.

106. **How to prevent race condition?**

Use synchronization mechanisms like locks or semaphores.

107. **What is busy waiting?**

Process repeatedly checks condition while wasting CPU.

108. **How to avoid busy waiting?**

Use blocking synchronization like semaphores.

109. **What is spinlock?**

Lock where process busy waits until it acquires lock.

110. **Use of spinlock?**

Efficient for short critical sections in SMP systems.

111. **What is cooperative multitasking?**

Processes voluntarily yield CPU.

112. **What is preemptive multitasking?**

OS interrupts to switch CPU between processes.

113. **What is cooperative vs preemptive multitasking?**

Cooperative = voluntary; Preemptive = enforced switching.

114. **What is kernel thread vs user thread?**

Kernel = managed by OS; User = managed by library.

115. **Advantages of kernel thread?**

True parallelism, better system control.

116. **Advantages of user thread?**

Fast creation, less overhead, flexible scheduling.

117. **What is thread synchronization?**

Coordinating threads to prevent conflicts in shared data.

118. **What is monitor in OS?**

High-level synchronization construct encapsulating locks and condition variables.

119. **What is condition variable?**

Used with monitors for threads to wait for certain condition.

120. **What is context switch overhead?**

CPU time wasted saving/restoring process state.

121. **How to reduce context switch overhead?**
Minimize frequent preemptions or increase time quantum.
122. **What is kernel preemption?**
OS can preempt running kernel tasks for responsiveness.
123. **What is cooperative kernel?**
Kernel code runs to completion without preemption.
124. **What is idle process?**
Process that runs when no other process is ready.
125. **What is process scheduling queue?**
Ready, waiting, and device queues for managing process states.
126. **What is dispatcher?**
Module that switches CPU to selected process.
127. **What is dispatch latency?**
Time taken to stop one process and start another.
128. **What is multiprocessor scheduling?**
Assigning processes among multiple CPUs efficiently.
129. **Types of multiprocessor scheduling?**
Asymmetric, symmetric, load sharing.
130. **What is real-time scheduling algorithm?**
Rate Monotonic, Earliest Deadline First for hard deadlines.
131. **What is rate monotonic scheduling (RMS)?**
Fixed priority based on periodic task frequency.
132. **What is earliest deadline first (EDF)?**
Dynamic priority scheduling based on task deadlines.
133. **Difference between RMS and EDF?**
RMS fixed priority; EDF dynamic priority.
134. **What is I/O scheduling?**
Optimizing order of disk access for performance.
135. **Types of I/O scheduling?**
FCFS, SSTF, SCAN, C-SCAN.

136. **What is SSTF?**

Shortest Seek Time First; selects nearest track request.

137. **What is SCAN scheduling?**

Disk arm moves like **elevator serving requests** in order.

138. **What is C-SCAN scheduling?**

Circular SCAN; returns to start after reaching end.

139. **What is FCFS disk scheduling?**

Serve **requests in arrival order**; simple but inefficient.

140. **What is file system?**

OS module managing storage and data organization.

141. **What is file?**

Collection of related data identified by name.

142. **What is file control block (FCB)?**

Metadata storing file attributes and location info.

143. **What is directory?**

Container mapping file names to **FCBs**.

144. **Types of file allocation?**

Contiguous, linked, indexed.

145. **What is contiguous allocation?**

Files stored in **consecutive disk blocks**.

146. **What is linked allocation?**

Files stored in **linked list** of disk blocks.

147. **What is indexed allocation?**

Index block contains **pointers to file blocks**.

148. **Advantages of indexed over linked allocation?**

Random access possible; no **pointer traversal** needed.

149. **What is paging in memory management?**

Dividing **memory** into **fixed-size frames** and **pages**.

150. **What is segmentation in memory management?**

Dividing memory into **variable-length logical segments**.

151. **What is memory fragmentation?**

Wasted memory due to non-contiguous allocation (internal or external).

152. **Difference between internal and external fragmentation?**

Internal = wasted inside allocated block; External = free scattered blocks.

153. **What is compaction?**

Combining free memory to form contiguous blocks to reduce fragmentation.

154. **What is buddy system in memory allocation?**

Memory divided recursively into halves to satisfy allocation requests.

155. **Advantages of buddy system?**

Fast allocation, easy merging, reduces external fragmentation.

156. **What is slab allocation?**

Pre-allocating caches of commonly used objects to reduce overhead.

157. **What is memory-mapped file?**

Maps file contents into process address space for fast I/O.

158. **What is demand segmentation?**

Loading only required segments of a process into memory.

159. **What is inverted page table?**

Single table indexed by frame containing process and page info.

160. **Advantages of inverted page table?**

Reduces memory overhead for large address spaces.

161. **What is TLB?**

Translation Lookaside Buffer caches recent page table entries.

162. **How TLB improves performance?**

Reduces time for virtual-to-physical address translation.

163. **What is effective memory access time?**

Average memory access considering TLB hits and misses.

164. **What is page replacement policy?**

Algorithm to decide which page to remove when memory full.

165. **What is optimal page replacement?**

Removes page that won't be used for longest time; theoretical best.

166. **What is FIFO page replacement?**

Removes oldest loaded page in memory.

167. **What is LRU page replacement?**

Removes least recently used page to minimize future faults.

168. **What is clock page replacement?**

Circular buffer approximation of LRU using reference bit.

169. **Difference between LRU and Clock algorithm?**

LRU precise but costly; Clock = efficient approximation.

170. **What is working set model?**

Tracks active pages of process to allocate memory efficiently.

171. **How does OS prevent thrashing using working set?**

Ensures each process gets enough frames to reduce page faults.

172. **What is demand-paged virtual memory?**

Pages loaded only when referenced, reducing unnecessary I/O.

173. **What is segmentation fault reason?**

Invalid memory access outside assigned segment.

174. **What is copy-on-write?**

Delay copying pages until process modifies them.

175. **Advantage of copy-on-write?**

Efficient memory use during fork or process duplication.

176. **What is shared memory mechanism?**

Multiple processes access same memory region for IPC.

177. **What are the benefits of shared memory over message passing?**

Faster, direct data access without kernel involvement.

178. **What is message queue IPC?**

Processes exchange structured messages using kernel queues.

179. **What is pipe in OS?**

Unidirectional communication channel between related processes.

180. **Difference between anonymous and named pipe?**

Anonymous = related processes; Named = unrelated processes.

181. **What is socket in OS?**

Endpoint for communication between processes across networks.

182. **Difference between stream and datagram socket?**

Stream = TCP reliable; Datagram = UDP connectionless.

183. **What is process synchronization problem?**

Ensuring correct sequencing of process execution on shared resources.

184. **What is race condition example?**

Two threads increment same counter simultaneously causing wrong result.

185. **What is Peterson's solution?**

Software algorithm ensuring mutual exclusion between two processes.

186. **What is bakery algorithm?**

Software solution for mutual exclusion for multiple processes using numbering.

187. **Difference between mutex and semaphore?**

Mutex = mutual exclusion lock; Semaphore = signaling/counting resource.

188. **What is monitor advantage?**

High-level construct providing automatic mutual exclusion and condition variables.

189. **What is spinlock usage?**

Busy-wait lock suitable for short critical sections in SMP.

190. **What is deadlock prevention?**

Ensure one necessary condition of deadlock cannot occur.

191. **What is deadlock avoidance?**

OS allocates resources only if it keeps system in safe state.

192. **What is deadlock detection?**

Identify circular wait cycles using resource allocation graph.

193. **How to recover from deadlock?**

Terminate process or preempt resources.

194. **Difference between prevention, avoidance, and detection?**

Prevention = block condition; Avoidance = safe allocation; Detection = identify & resolve.

195. **What is multiprocessor scheduling?**

Assigning processes among multiple CPUs efficiently.

196. **Types of multiprocessor scheduling?**
Asymmetric, symmetric, and load sharing.
197. **What is asymmetric multiprocessing?**
One CPU handles OS, others execute user processes.
198. **What is symmetric multiprocessing?**
All CPUs share equal access to OS and tasks.
199. **Advantages of SMP over AMP?**
Scalability, fault tolerance, and parallel execution.
200. **What is real-time scheduling?**
Algorithm meeting strict timing constraints for tasks.
201. **Hard vs soft real-time scheduling?**
Hard = deadlines critical; Soft = occasional miss tolerable.
202. **What is Rate Monotonic Scheduling?**
Fixed priority; higher frequency tasks get higher priority.
203. **What is Earliest Deadline First scheduling?**
Dynamic priority based on task deadline.
204. **Difference between RMS and EDF?**
RMS fixed priority; EDF dynamic priority based on deadline.
205. **What is I/O scheduling?**
Ordering disk requests to reduce seek time and improve throughput.
206. **What is SSTF disk scheduling?**
Selects request closest to current head position.
207. **What is SCAN disk scheduling?**
Disk arm moves in one direction serving requests, then reverses.
208. **What is C-SCAN scheduling?**
Circular SCAN; after end, returns to start without reversing.
209. **Difference between SCAN and C-SCAN?**
SCAN = bi-directional; C-SCAN = unidirectional, uniform wait.
210. **What is file system?**
OS module organizing data storage and access.

211. **What is inode?**

Metadata structure storing file attributes and block info.

212. **What is journaling file system?**

Maintains log to recover from crashes without data loss.

213. **What is directory structure?**

Organizes files using hierarchical tree structure.

214. **What is contiguous file allocation?**

Files occupy consecutive blocks for fast access.

215. **What is linked file allocation?**

Files stored as linked list of blocks; no contiguous requirement.

216. **What is indexed file allocation?**

Index block points to all file blocks for random access.

217. **Advantages of indexed over linked allocation?**

Random access is possible, traversal not needed.

218. **What is journaling in OS?**

Log of file system changes for fast crash recovery.

219. **What is log-structured file system?**

Sequential writes to log to improve write performance.

220. **What is virtual file system (VFS)?**

Abstract interface allowing multiple file systems to be accessed uniformly.

221. **What is memory-mapped I/O?**

Map device registers to process address space for direct access.

222. **What is DMA?**

Direct Memory Access allows devices to read/write without CPU.

223. **Advantage of DMA over programmed I/O?**

Reduces CPU involvement, faster data transfer.

224. **What is interrupt-driven I/O?**

Device interrupts CPU when ready instead of polling.

225. **What is polling?**

CPU repeatedly checks device status for readiness.

226. **Difference between polling and interrupt-driven I/O?**

Polling = CPU active waiting; Interrupt = CPU notified asynchronously.

227. **What is spooling?**

Process queues I/O requests to a buffer for sequential processing.

228. **What is booting?**

Process of starting OS after powering on computer.

229. **Difference between cold boot and warm boot?**

Cold = power off then on; Warm = restart without power cycle.

230. **What is process hierarchy?**

Parent-child relationship among processes for creation and termination.

231. **What is fork() system call?**

Creates new process duplicating parent.

232. **What is exec() system call?**

Replaces process memory with new program.

233. **Difference between fork() and exec()?**

fork = duplicate; exec = replace process image.

234. **What is wait() system call?**

Parent waits for child process termination.

235. **What is exit() system call?**

Terminates current process and returns status.

236. **What is zombie process?**

Child terminated but parent hasn't read exit status.

237. **What is orphan process?**

Child whose parent terminated; adopted by init process.

238. **What is signal in OS?**

Notification sent to process to handle asynchronous events.

239. **What is signal handler?**

Function executed in response to a signal.

240. **What is pipe in IPC?**

Unidirectional data channel between processes.

241. **Difference between anonymous and named pipe?**

Anonymous = related processes; Named = unrelated processes.

242. **What is socket IPC?**

Endpoint for communication between processes over network.

243. **What is thread in OS?**

Lightweight process sharing memory of parent process.

244. **What is multithreading advantage?**

Faster execution, better resource sharing, responsiveness.

245. **Difference between user-level and kernel-level thread?**

User = fast, managed by library; Kernel = slower, OS-managed.

246. **What is thread pool?**

Pre-created threads reused for executing tasks.

247. **What is process scheduling?**

Mechanism to decide order of CPU allocation to processes.

248. **What is dispatcher?**

Switches CPU to selected process from ready queue.

249. **What is dispatch latency?**

Time taken to stop one process and start another.

250. **What is preemptive scheduling advantage?**

Ensures high-priority tasks get CPU, improves responsiveness.

251. **What is cooperative scheduling?**

Processes voluntarily yield CPU.

252. **What is multilevel queue scheduling?**

Processes divided into priority queues with separate scheduling.

253. **What is multilevel feedback queue?**

Processes move between queues based on CPU burst behavior.

254. **What is aging in scheduling?**

Gradually increases priority of waiting processes to prevent starvation.

255. **What is starvation?**

Process waits indefinitely due to low priority.

256. **What is CPU-bound vs I/O-bound process?**

CPU-bound = computation heavy; I/O-bound = waiting heavy.

257. **How OS improves CPU utilization for I/O-bound processes?**

Multitasking with overlapping I/O and CPU work.

258. **What is semaphore in OS?**

Integer variable used for process synchronization.

259. **Difference between binary and counting semaphore?**

Binary = 0/1; Counting = integer counter.

260. **What is mutex?**

Lock allowing only one process in critical section.

261. **Difference between mutex and semaphore?**

Mutex = mutual exclusion; Semaphore = signaling/counting.

262. **What is condition variable?**

Synchronization primitive allowing threads to wait for condition.

263. **What is critical section problem?**

Ensuring one process accesses shared resource at a time.

264. **What is producer-consumer problem?**

Coordination of producer and consumer sharing buffer.

265. **What is solution for producer-consumer problem?**

Semaphores to track empty/full slots.

266. **What is reader-writer problem?**

Multiple readers allowed, writers require exclusive access.

267. **What is solution for reader-writer problem?**

Semaphores or mutex to prevent race conditions.

268. **What is dining philosophers problem?**

Demonstrates deadlock in resource allocation.

269. **What is solution for dining philosophers problem?**

Limit concurrent philosophers or enforce resource ordering.

270. **What is kernel mode vs user mode?**

Kernel = full access; User = restricted access.

271. **Why separate kernel and user mode?**
Protect OS and prevent accidental hardware corruption.
272. **What is system call?**
Interface for applications to request OS services.
273. **What is fork() and exec() relation?**
fork creates process; exec replaces its code with new program.
274. **What is zombie and orphan process handling?**
Orphan adopted by init; Zombie parent reads exit status.
275. **What is context switch?**
Save state of current process, load next process.
276. **Why context switching costly?**
Saves/restores registers, memory maps, and PC.
277. **What is interrupt?**
Signal to CPU to pause and handle event.
278. **Difference between hardware and software interrupt?**
Hardware = device; Software = program-generated.
279. **What is trap?**
Synchronous interrupt to request OS service.
280. **What is polling?**
CPU repeatedly checks device readiness.
281. **Advantages of interrupt over polling?**
Reduces CPU idle waiting, faster response.
282. **What is DMA?**
Direct Memory Access bypassing CPU for I/O transfer.
283. **Advantage of DMA?**
Improves throughput, frees CPU for computation.
284. **What is multilevel feedback queue scheduling?**
Dynamically adjusts process priority based on behavior.
285. **What is aging in multilevel queue?**
Increases priority of waiting process to prevent starvation.

286. **What is process control block (PCB) fields?**

PID, state, registers, program counter, memory info.

287. **What is thread control block (TCB)?**

Stores thread-specific info like ID, state, registers.

288. **Difference between PCB and TCB?**

PCB = process info; TCB = thread info within process.

289. **What is critical section entry protocol?**

Request, execute, and release mutual exclusion.

290. **What is busy waiting vs blocking?**

Busy waiting spins CPU; blocking suspends process until condition.

291. **What is cooperative kernel?**

Kernel code runs to completion without preemption.

292. **What is preemptive kernel?**

Kernel code can be interrupted for higher-priority process.

293. **What is idle process?**

Runs when no other ready process exists.

294. **What is CPU scheduling queue?**

Ready queue holding processes waiting for CPU.

295. **What is dispatch latency?**

Time taken by dispatcher to switch CPU between processes.

296. **What is thread pool advantage?**

Reduces overhead of creating/destroying threads repeatedly.

297. **Difference between process and thread?**

Process = own memory; Thread = shares memory within process.

298. **What is preemptive vs non-preemptive scheduling?**

Preemptive = OS interrupts; Non-preemptive = runs till done.

299. **What is starvation and aging?**

Starvation = indefinite wait; Aging = gradual priority increase.

300. **What is multiprogramming and multitasking difference?**

Multiprogramming = maximize CPU; Multitasking = multiple interactive tasks.

301. **What is cooperative threading vs concurrent threading?**

Cooperative threads yield CPU voluntarily; concurrent threads run simultaneously on multiple cores.

302. **What is priority inversion?**

Low-priority task holds resource needed by high-priority task, causing delay.

303. **How to solve priority inversion?**

Use priority inheritance: temporarily raise low-priority task's priority.

304. **What is real-time clock in OS?**

Hardware timer providing accurate time and generating interrupts for scheduling.

305. **What is kernel preemption latency?**

Time from interrupt arrival to kernel responding; affects real-time responsiveness.

306. **What is soft vs hard links in file system?**

Soft link = symbolic pointer to file; Hard link = additional directory entry pointing to inode.

307. **What is demand segmentation with paging?**

Combines paging and segmentation; only required pages of a segment are loaded.

308. **What is copy-on-reference (COR)?**

Variant of copy-on-write used in multithreading to optimize memory usage.

309. **What is asynchronous vs synchronous I/O?**

Synchronous = process waits for I/O; Asynchronous = process continues while I/O completes.

310. **What is kernel-level scheduling vs user-level scheduling?**

Kernel schedules processes/threads globally; user-level scheduler manages threads within process.

311. **What is multiprocessor affinity?**

Process/thread prefers running on specific CPU to improve cache performance.

312. **What is page coloring?**

Technique to reduce cache conflicts by controlling which physical pages map to which cache lines.